

CYBR 470 project

Reverse Engineering Russian Applications
Presented By: Calvin Ruch



Context - Papers from before

- *"Large Russian companies have been instructed by the Ministry of Digital Development to engage in censorship and surveillance of users. Starting April 15, 2026, they will be required to restrict access to users who have VPNs enabled on their devices."*
 - 463 possible affected Android applications
- Links:
 - <https://github.com/net4people/bbs/issues/605>
 - https://files.rks.global/russian_apps_search_for_vpn_en.pdf

Context - What they found

- They found 22/30 Russian apps detect VPNs
 - They found 19/30 Russian apps send status to server
 - 29 out of 30 apps check for root access (all except Yandex Go)
 - 24 out of 30 request microphone access
 - 28 out of 30 request camera access
 - 24 out of 30 send the list of installed apps to their servers
 - 49 out of 68 surveillance checks operate without user interaction — immediately after installation
-
- Analysis link: [Database link](#)

Tools Used

- Everything was done in a Linux environment
- A combination of Python and Bash was used
- Jadx-1.5.5
 - For Java decompilation
- Apktool_3.0.2.jar
 - For Smali decompilation
- Ripgrep
 - For searching for strings inside the smali files

Overview of what I did

- Preparing for project
 - Find all of the apps on Play store and Rstore
 - Create custom scripts to extract all apps from the Android phone
 - Safe handling
- Analysis of APK
 - Using both apktool and Jadx decompiled APKs for analysis
 - Next slide shows workflow
 - This is where I found the fingerprints
- Create auto analysis
 - Use combination of ripgrep, bash and python to automate this analysis and print readable results

```
public final String t() {  
    return "Android " + Build.VERSION.RELEASE + " (SDK " + Build.VERSION.SDK_INT + ")";  
}
```

```
method public final t()Ljava/lang/String;
    .locals 4

    .line 1
    sget-object v0, Landroid/os/Build$VERSION;.>RELEASE:Ljava/lang/String;

    .line 2
    .line 3
    sget v1, Landroid/os/Build$VERSION;.>SDK_INT:I

    .line 4
    .line 5
    new-instance v2, Ljava/lang/StringBuilder;

    .line 6
    .line 7
    invoke-direct {v2}, Ljava/lang/StringBuilder;.><init>()V

    .line 8
    .line 9
    .line 10
    const-string v3, "Android "

    .line 11
    .line 12
    invoke-virtual {v2, v3}, Ljava/lang/StringBuilder;.>append(Ljava/lang/String;)Ljava/lang/StringBuilder;

    .line 13
    .line 14
    .line 15
    invoke-virtual {v2, v0}, Ljava/lang/StringBuilder;.>append(Ljava/lang/String;)Ljava/lang/StringBuilder;

    .line 16
    .line 17
    .line 18
    const-string v0, " (SDK "

    .line 19
    .line 20
    invoke-virtual {v2, v0}, Ljava/lang/StringBuilder;.>append(Ljava/lang/String;)Ljava/lang/StringBuilder;

    .line 21
    .line 22
    .line 23
    invoke-virtual {v2, v1}, Ljava/lang/StringBuilder;.>append(I)Ljava/lang/StringBuilder;

    .line 24
    .line 25
    .line 26
    const-string v0, ")"

    .line 27
    .line 28
    invoke-virtual {v2, v0}, Ljava/lang/StringBuilder;.>append(Ljava/lang/String;)Ljava/lang/StringBuilder;

    .line 29
    .line 30
    .line 31
    invoke-virtual {v2}, Ljava/lang/StringBuilder;.>toString()Ljava/lang/String;

    .line 32
    .line 33
    .line 34
    move-result-object v0

    .line 35
    return-object v0
end method
```

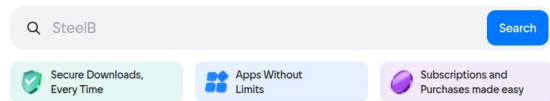
Overview of what I accomplished

- I got somewhat similar results
- I may have added too many arbitrary fingerprints (may need refining)
- I found all of the apps I was searching through to surveil VPNs
- Most were found in other categories too.

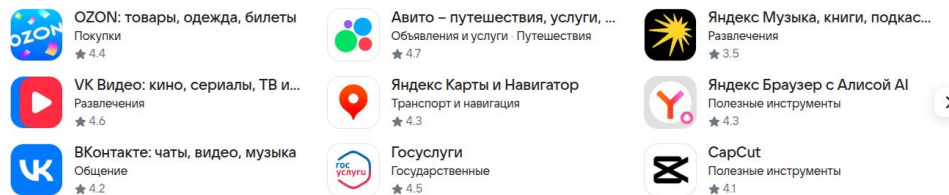
Preparing for project

Official Android App Store

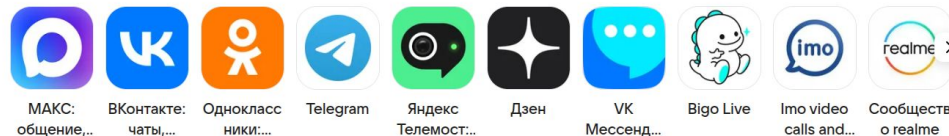
Your reliable way to Top Games and Apps



Популярное



Социальные сети и мессенджеры



Detecting Surveillance in 30 Popular Russian Apps

Method: static APK analysis (decompilation, search across 68 surveillance checkpoints in 12 categories)

Sample size: 30 apps from RuStore and Google Play



Preparing for Project cont.

```
#!/bin/bash
cat scripts/textLists/CalsArt.txt

# Pulling the apps off the phone section
adb shell pm list packages -f | grep "package:/data" > scripts/textLists/apklist.txt
ls pulledAps > scripts/textLists/installedAps.txt
python3 scripts/APKparser.py > scripts/textLists/apklistParsed.txt
while read -r line; do
    APKpath=$(echo "$line" | awk '{print $1}')
    APKname=$(echo "$line" | awk '{print $2}')
    echo "Installing :$APKname"
    adb pull "$APKpath" "./pulledAps/$APKname.apk"
done < scripts/textLists/apklistParsed.txt

# Unpacking all of the APKs
ls pulledAps > scripts/textLists/installedAps.txt
while read -r line; do
    curAPK=$(echo "pulledAps/$line")
    echo "Unpacking $curAPK"
    ./scripts/pullAPK.sh $curAPK
done < scripts/textLists/installedAps.txt
```

```
lachurc@lachurc-Nitro:~/Documents/school/CYBR470/project$ ls pulledAps/
com.avito.android
com.avito.android.apk
com.ertelecom.agent
com.ertelecom.agent.apk
com.google.android.odad
com.google.android.odad.apk
com.google.android.youtube
com.google.android.youtube.apk
com.idamob.tinkoff.android
com.idamob.tinkoff.android.apk
com.wildberries.ru.apk
com.wildberries.ru.apk
com.topjohnwu.magisk
com.topjohnwu.magisk.apk
com.uma.musicvk
com.uma.musicvk.apk
com.vkontakte.android
com.vkontakte.android.apk
com.vk.vkvideo
com.vk.vkvideo.apk
com.wildberries.ru.apk
com.wildberries.ru.apk
```

```
com.yandex.browser
com.yandex.browser.apk
ru.alfabank.mobile.android
ru.alfabank.mobile.android.apk
ru.beru.android
ru.beru.android.apk
ru.dublgis.dgismobile
ru.dublgis.dgismobile.apk
ru.foodfox.client
ru.foodfox.client.apk
ru.kinopoisk
ru.kinopoisk.apk
ru.mail.mailapp
ru.mail.mailapp.apk
ru.megafon.mlk
ru.megafon.mlk.apk
ru.megamarket.marketplace
ru.megamarket.marketplace.apk
ru.mts.mymts
ru.mts.mymts.apk
```

```
with open("scripts/textLists/installedAps.txt","r") as f1:
    uapks = f1.read().splitlines()

for i in range(len(uapks)):
    uapks[i] = uapks[i].removesuffix(".apk")

with open("scripts/textLists/apklist.txt", "r") as f:
    apks = f.read().splitlines()

for apk in apks:
    apkPath=apk.split(":")[1].split("apk=")[0]
    apkName=apk.split("=")[-1]
    if apk.split("=")[-1] in skipList:
        continue
    elif apk.split("=")[-1] in uapks:
        continue
    else:
        print(f"{apkPath}apk {apkName}")
```

```
ru.nspk.mirpay
ru.nspk.mirpay.apk
ru.ok.android
ru.ok.android.apk
ru.oneme.app
ru.oneme.app.apk
ru.ozon.app.android
ru.ozon.app.android.apk
ru.rostel
ru.rostel.apk
ru.rutube.app
ru.rutube.app.apk
ru.sbcs.store
ru.sbcs.store.apk
ru.sberbankmobile
ru.sberbankmobile.apk
ru.vk.store
ru.vk.store.apk
ru.vtb24.mobilebanking.android
ru.vtb24.mobilebanking.android.apk
ru.yandex.music
ru.yandex.music.apk
ru.yandex.taxi
ru.yandex.taxi.apk
ru.yandex.yandexmaps
ru.yandex.yandexmaps.apk
ru.zen.android
ru.zen.android.apk
```

Analysis - intial

All of my manual analysis came from VTB online

- Used a combination of ripgrep and vim to manually look through the smali files.
- Below are the interesting ones I found and in the order I found them:
 - NetworkSettingsDto.java
 - lv52.java
 - DiagnosticResultDto.java
 - NetworkInfo.java
 - NetworkInfoCollector.java

Analysis - lv52.java and NetworkSettingsDto.java

A couple of the interesting lines stood out to me when digging around:

```
AdapterStatusDto adapterStatusDto = new AdapterStatusDto(networkInfo.getIsMobileInternetOn(),  
    networkInfo.getIsWifiInternetOn(), networkInfo.getSignalLevel().name(), networkInfo.getOperatorName());
```

```
String systemName = networkInfo.getSystemName();
```

```
String deviceName = networkInfo.getDeviceName();
```

```
ek52.a.w("NetworkDiagnostic").a("Все данные собраны, начинается конвертация для  
отправки на сервер", new Object[0]);
```

The screenshot shows a Google Translate interface. At the top, there are language selection buttons: "Russian - Detected", "English", "Spanish", and "French". Below this, the Russian text "Все данные собраны, начинается конвертация для отправки на сервер" is entered into a text box. To the right of the text box is a close button (X). Below the text box, there is a small text box containing the transliterated Russian text "Vse dannyye sobrany, nachinayetsya konvertatsiya dlya otpravki na server". Below this is a microphone icon and a speaker icon. To the right of the Russian text box is a large light blue box containing the English translation "All data has been collected; conversion for transmission to the server is starting." To the right of the English text box is a star icon. Below the English text box, there are icons for Google, a share icon, and a feedback icon. At the bottom right of the interface, there is a link that says "Send feedback".

[Send feedback](#)

Analysis - NetworkInfoCollector.java

Function	new name	Functionality	Evidence/ Code
a	init	constructs the class	
k	setup	sets up the context library and NetworkInfoCollector object	
l	getDeviceInfo	This will get the device info	Build.MANUFACTURER + ProductCell.DELIMITER + Build.MODEL
m	getDNS	This will get the DNS information	
n	?	has conditional statement	return r(context) ? v(context) : u(context, continuation);
p	getProxy	This will get the info about proxies being used	linkProperties.getHttpProxy()
q	getVPN	This will get the VPN info	
r	getWIFI	This will get the wifi information	
s	getPhoneOperator	This will get the phone operator name (translated the russian)	
t	getAndroid version	This will get the build version and sdk version?	"Android " + Build.VERSION.RELEASE + " (SDK " + Build.VERSION.SDK_INT + ")"
v	getWifiSignal	This will get the wifi signal level (translated the russian)	

Breakdown of all of the functions - Part of my documentation

I used functions l, p, q, r, s, t and v.

You might notice a couple functions are missing: o and u

- Jadx struggled to decompile these
- Visible in smali decompiled version.

Auto analysis tools

I created a few scripts for this portion of the project that work.

- APKtermSearch.sh
 - This holds all of the Ripgrep commands I need, for manual searching or working with another script.
- autoTS.sh
 - This is the main workhorse of the auto analysis this script will call everything else.
- rgLParser.py
 - This is used to parse through the lists that ripgrep creates, this will count the amount of fingerprint occurrences a in each decompiled APK.
- resultsExtractor.py
 - This will print all of the results garnered by autoTS.sh .
- termAdder.sh
 - This is an interactive UI that will add terms to the fingerprint files, this also allows for two terms to be entered, such that results match both terms.
 - This uses custom delimiter, if you look through the text files after running you will see '!<>!' between terms.

Auto Analysis tool - APKtermSearch

```
#!/bin/bash
# This program is used to find all of the smali files in an extracted apk
# A more efficient version of this could be made but ripgrep made for ea
#
# Barely any error checking - enter the correct directory or it will run

# Make sure there is 3 args
# $1 = extracted apk directory to search
# $2 = search term
# $3 = option
# - 1 = get all file names - only smali files
# - 2 = get all file names
# - 3 = get all of the text (normal ripgrep style)
# - 4 = Use custom chain of commands for script
#
# Ripgrep flags
# -F = fixed string search, not using regex to search for strings
# -l = Get the File name that the matching string was in (option 3 does
# -- = I was having issues and this was supposed to help with the parsin

# Sed command
# 's|/.*||' - This will Chop everything off after '/', so I can get the
# 's#!<!.*##' - This will get the first part of my custom delimiter
# 's^.*!<!.*##' - This will get the second part of my custom delimiter

# Error handling if there isn't 3 arguments
if [ $# -ne 3 ]; then
    echo "Arg 1: Directory to search (use . to search everything)"
    echo "Arg 2: Search term to look for"
    echo "Arg 3: Option to use (Default is 1)"
    echo "Arg 4: get APKs that contain the search term"
    exit
fi

# This will get you into the pulled APKs directory
cd "pulledApks/$1"

# This part is for the actual ripgrep portion
if [ $3 -eq '1' ]; then
    rg -l -F -- "$2" | grep ".smali"
elif [ $3 -eq '2' ]; then
    rg -l -F -- "$2"
elif [ $3 -eq '3' ]; then
    rg -F -- "$2"
elif [ $3 -eq '4' ]; then
    touch ../scripts/textLists/rgList.txt
    rg -l -F -- "$2" | grep ".smali" | sed 's|/.*||' > ../scripts/textLists/rgList.txt
    cd ..
    python3 scripts/rgLParser.py
elif [ $3 -eq '5' ]; then
    string1=$(echo "$2" | sed 's#!<!.*##')
    string2=$(echo "$2" | sed 's^.*!<!.*##')
    # This is a custom chain of ripgrep commands to search for two terms.
    rg -l -F -- "$string1" | xargs -r rg -l -F -- "$string2" | sed 's|/.*||' > ../scripts/textLists/rgList.txt
    cd ..
    python3 scripts/rgLParser.py
fi
```

Auto Analysis tool - autoTS

```
verboseMessage() {  
    if [ "$2" -eq 1 ]; then  
        echo "$1"  
    fi  
}  
  
# Custom function to print to the Finger Print text file  
printFile() {  
    echo "|$1" >> scripts/textLists/FingerPrint.txt  
}  
  
clear
```

```
# Get all of the finger print files  
ls fingerprint > scripts/textLists/fingerprintFiles.txt  
  
# Clear the fingerprint file  
> scripts/textLists/FingerPrint.txt  
  
# Iterate through all of the fingerprint files  
while read -r fingerprintFile; do  
  
    # Print the File name to the Fingerprint.txt  
    printFile "$fingerprintFile"  
    verboseMessage "Searching for fingerprint file: $fingerprintFile" "$verbose"  
  
    # Read current fingerprint file and perform the term search on each  
    # Then save the result to a text file for analysis later in script  
    while read -r line; do  
        verboseMessage "Searching for term: $line" "$verbose"  
        printFile "$line"  
  
        if [[ "$line" == *"!<>!*" ]]; then  
            ./scripts/APKtermSearch.sh . "$line" 5 < /dev/null  
        else  
            # Perform the search on the APK list - Concat result onto text file  
            ./scripts/APKtermSearch.sh . "$line" 4 < /dev/null  
        fi  
        cat scripts/textLists/rgwList.txt >> scripts/textLists/FingerPrint.txt  
    done < "fingerprint/$fingerprintFile"  
done < scripts/textLists/fingerprintFiles.txt  
  
#cat scripts/textLists/FingerPrint.txt  
python3 scripts/resultsExtractor.py
```


Auto Analysis tool - rgLParser.py

```
def main():
    rgFileR = 'scripts/textLists/rgList.txt'
    rgFileW = 'scripts/textLists/rgWList.txt'

    # Read the temporary input file
    with open(rgFileR, 'r') as f:
        file = f.read()

    # Create empty dictionary
    results = {}

    # Count the occurrences of each app and store it into a dictionary
    for app in file.splitlines():
        app = app.strip()
        if app in results:
            results[app] += 1
        else:
            results[app] = 1

    # Write all of the results to the output file
    with open(rgFileW, 'w') as f:
        for app, count in results.items():
            if len(sys.argv) > 1 and sys.argv[1] == "v": print(f"{app}: {count}")
            f.write(f"{app}: {count}\n")
```


Auto Analysis tool - resultsExtractor

- If you want to you can look at the code on my github
 - It was hacked together in a couple of hours and it doesn't look pretty
 - It does work though and it has a verbose mode so more information can be seen

Auto analysis demo

Now for the demo of the program:

Auto analysis demo

Now for the demo of the program:

- I would do a demo of the program

Auto analysis demo

Now for the demo of the program:

- I would do a demo of the program
 - Not the most efficient (super slow)

Auto analysis demo

Now for the demo of the program:

- I would do a demo of the program
 - Not the most efficient (super slow)
 - Murphy's law might apply to me right now.

Auto analysis demo

Now for the demo of the program:

- I would do a demo of the program
 - Not the most efficient (super slow)
 - Murphy's law might apply to me right now.
 - What I will do is show the output of the program

QAL APK Extractor

```
Searching for fingerprint file: cellularF.txt
Searching for term: Landroid/telephony/TelephonyManager;->getNetworkOperatorName()Ljava/lang/String;
Searching for term: getSystemService(Ljava/lang/String)!<>!phone
Searching for term: Landroid/telephony/TelephonyManager;
Searching for fingerprint file: deviceF.txt
Searching for term: Landroid/os/Build;->MANUFACTURER:Ljava/lang/String;
Searching for term: Landroid/os/Build;->MODEL:Ljava/lang/String;
Searching for term: Landroid/os/Build$VERSION;->RELEASE:Ljava/lang/String;
Searching for term: Landroid/os/Build$VERSION;->SDK_INT:I
Searching for fingerprint file: miscF.txt
Searching for term: org.torproject.torbrowser
Searching for fingerprint file: proxyF.txt
Searching for term: Landroid/net/LinkProperties!<>!getHttpProxy()Landroid/net/ProxyInfo
Searching for term: getHttpProxy()
Searching for term: http.proxyHost
Searching for fingerprint file: vpnF.txt
Searching for term: Landroid/net/ConnectivityManager;->getNetworkCapabilities(Landroid/net/Network;)Landroid/net/NetworkCapabilities
Searching for term: Landroid/net/ConnectivityManager;->getActiveNetwork()Landroid/net/Network;
Searching for term: Landroid/net/NetworkCapabilities;->hasTransport(I)Z!<>!const/4 v1, 0x4
Searching for term: TRANSPORT_VPN
Searching for term: tun0
Searching for term: /proc/net/tcp
Searching for fingerprint file: wifiF.txt
Searching for term: Landroid/net/wifi/WifiManager;->getConnectionInfo()Landroid/net/wifi/WifiInfo;
Searching for term: Landroid/net/wifi/WifiInfo;->getRssi()I
```

QAL APK Extractor

```
Searching for fingerprint file: cellularF.txt
Searching for term: Landroid/telephony/TelephonyManager;->getNetworkOperatorName()Ljava/lang/String;
Searching for term: getSystemService(Ljava/lang/String)!<>!phone
Searching for term: Landroid/telephony/TelephonyManager;
Searching for fingerprint file: deviceF.txt
Searching for term: Landroid/os/Build;->MANUFACTURER:Ljava/lang/String;
Searching for term: Landroid/os/Build;->MODEL:Ljava/lang/String;
Searching for term: Landroid/os/Build$VERSION;->RELEASE:Ljava/lang/String;
Searching for term: Landroid/os/Build$VERSION;->SDK_INT:I
Searching for fingerprint file: miscF.txt
Searching for term: org.torproject.torbrowser
Searching for fingerprint file: proxyF.txt
Searching for term: Landroid/net/LinkProperties!<>!getHttpProxy()Landroid/net/ProxyInfo
Searching for term: getHttpProxy()
Searching for term: http.proxyHost
Searching for fingerprint file: vpnF.txt
Searching for term: Landroid/net/ConnectivityManager;->getNetworkCapabilities(Landroid/net/Network;)Landroid/net/NetworkCapabilities
Searching for term: Landroid/net/ConnectivityManager;->getActiveNetwork()Landroid/net/Network;
Searching for term: Landroid/net/NetworkCapabilities;->hasTransport(I)Z!<>!const/4 v1, 0x4
Searching for term: TRANSPORT_VPN
Searching for term: tun0
Searching for term: /proc/net/tcp
Searching for fingerprint file: wifiF.txt
Searching for term: Landroid/net/wifi/WifiManager;->getConnectionInfo()Landroid/net/wifi/WifiInfo;
Searching for term: Landroid/net/wifi/WifiInfo;->getRssi()I
Searching for term: Landroid/net/wifi/WifiManager;->getConnectionInfo()Landroid/net/wifi/WifiInfo;!<>!Landroid/net/wifi/WifiInfo;->getRssi()I
```


CALE APK EXTRACTOR

```
Searching for fingerprint file: cellularF.txt
Searching for term: Landroid/telephony/TelephonyManager;->getNetworkOperatorName()Ljava/lang/String;
Searching for term: getSystemService(Ljava/lang/String)!<>!phone
Searching for term: Landroid/telephony/TelephonyManager;
Searching for fingerprint file: deviceF.txt
Searching for term: Landroid/os/Build;->MANUFACTURER:Ljava/lang/String;
Searching for term: Landroid/os/Build;->MODEL:Ljava/lang/String;
Searching for term: Landroid/os/Build$VERSION;->RELEASE:Ljava/lang/String;
Searching for term: Landroid/os/Build$VERSION;->SDK_INT:I
Searching for fingerprint file: miscF.txt
Searching for term: org.torproject.torbrowser
Searching for fingerprint file: proxyF.txt
Searching for term: Landroid/net/LinkProperties!<>!getHttpProxy()Landroid/net/ProxyInfo
Searching for term: getHttpProxy()
Searching for term: http.proxyHost
Searching for fingerprint file: vpnF.txt
Searching for term: Landroid/net/ConnectivityManager;->getNetworkCapabilities(Landroid/net/Network;)Landroid/net/NetworkCapabilities
Searching for term: Landroid/net/ConnectivityManager;->getActiveNetwork()Landroid/net/Network;
Searching for term: Landroid/net/NetworkCapabilities;->hasTransport(I)Z!<>!const/4 v1, 0x4
Searching for term: TRANSPORT_VPN
Searching for term: tun0
Searching for term: /proc/net/tcp
Searching for fingerprint file: wifiF.txt
Searching for term: Landroid/net/wifi/WifiManager;->getConnectionInfo()Landroid/net/wifi/WifiInfo;
Searching for term: Landroid/net/wifi/WifiInfo;->getRssi()I
Searching for term: Landroid/net/wifi/WifiManager;->getConnectionInfo()Landroid/net/wifi/WifiInfo;!<>!Landroid/net/wifi/WifiInfo;->getRssi()I
Searching for term: Landroid/net/NetworkCapabilities;->hasTransport(I)Z!<>!const/4 v1, 0x1
```

Auto analysis demo

There is at least 2 minutes in between each of the term searches

- Not reasonable for a 30 minute presentation

ONLY APK EXTRACTOR

```
Searching for fingerprint file: cellularF.txt
Searching for term: Landroid/telephony/TelephonyManager; ->getNetworkOperatorName()Ljava/lang/String;
Searching for term: getSystemService(Ljava/lang/String)I!< !phone
Searching for term: Landroid/telephony/TelephonyManager;
Searching for fingerprint file: deviceF.txt
Searching for term: Landroid/os/Build; ->MANUFACTURER:Ljava/lang/String;
Searching for term: Landroid/os/Build; ->MODEL:Ljava/lang/String;
Searching for term: Landroid/os/Build$VERSION; ->RELEASE:Ljava/lang/String;
Searching for term: Landroid/os/Build$VERSION; ->SDK_INT:I
Searching for fingerprint file: miscF.txt
Searching for term: org.torproject.torbrowser
Searching for fingerprint file: proxyF.txt
Searching for term: Landroid/net/LinkProperties!< !getHttpProxy()Landroid/net/ProxyInfo
Searching for term: getHttpProxy()
Searching for term: http.proxyHost
Searching for fingerprint file: vpnF.txt
Searching for term: Landroid/net/ConnectivityManager; ->getNetworkCapabilities(Landroid/net/Network;)Landroid/net/NetworkCapabilities
Searching for term: Landroid/net/ConnectivityManager; ->getActiveNetwork()Landroid/net/Network;
Searching for term: Landroid/net/NetworkCapabilities; ->hasTransport(I)Z!< !const/4 v1, 0x4
Searching for term: TRANSPORT_VPN
Searching for term: tun0
Searching for term: /proc/net/tcp
Searching for fingerprint file: wifiF.txt
Searching for term: Landroid/net/wifi/WifiManager; ->getConnectionInfo()Landroid/net/wifi/WifiInfo;
Searching for term: Landroid/net/wifi/WifiInfo; ->getRssi()I
Searching for term: Landroid/net/wifi/WifiManager; ->getConnectionInfo()Landroid/net/wifi/WifiInfo; !< !Landroid/net/wifi/WifiInfo; ->getRssi()I
Searching for term: Landroid/net/NetworkCapabilities; ->hasTransport(I)Z!< !const/4 v1, 0x1
Searching for term: Landroid/net/ConnectivityManager; ->getNetworkCapabilities(Landroid/net/Network;)Landroid/net/NetworkCapabilities; !< !Landroid/net/NetworkCapabilities; ->hasTransport(I)Z
```

CALLS APK EXTRACTOR

```
Searching for fingerprint file: cellularF.txt
Searching for term: Landroid/telephony/TelephonyManager;-->getNetworkOperatorName()Ljava/lang/String;
Searching for term: getSystemService(Ljava/lang/String)!<=>!phone
Searching for term: Landroid/telephony/TelephonyManager;
Searching for fingerprint file: deviceF.txt
Searching for term: Landroid/os/Build;-->MANUFACTURER:Ljava/lang/String;
Searching for term: Landroid/os/Build;-->MODEL:Ljava/lang/String;
Searching for term: Landroid/os/Build$VERSION;-->RELEASE:Ljava/lang/String;
Searching for term: Landroid/os/Build$VERSION;-->SDK_INT:I
Searching for fingerprint file: miscF.txt
Searching for term: org.torproject.torbrowser
Searching for fingerprint file: proxyF.txt
Searching for term: Landroid/net/LinkProperties!<=>!getHttpProxy()Landroid/net/ProxyInfo
Searching for term: getHttpProxy()
Searching for term: http.proxyHost
Searching for fingerprint file: vpnF.txt
Searching for term: Landroid/net/ConnectivityManager;-->getNetworkCapabilities(Landroid/net/Network;)Landroid/net/NetworkCapabilities
Searching for term: Landroid/net/ConnectivityManager;-->getActiveNetwork()Landroid/net/Network;
Searching for term: Landroid/net/NetworkCapabilities;-->hasTransport(I)Z!<=>!const/4 v1, 0x4
Searching for term: TRANSPORT_VPN
Searching for term: tun0
Searching for term: /proc/net/tcp
Searching for fingerprint file: wifiF.txt
Searching for term: Landroid/net/wifi/WifiManager;-->getConnectionInfo()Landroid/net/wifi/WifiInfo;
Searching for term: Landroid/net/wifi/WifiInfo;-->getRssi()I
Searching for term: Landroid/net/wifi/WifiManager;-->getConnectionInfo()Landroid/net/wifi/WifiInfo;!<=>!Landroid/net/wifi/WifiInfo;-->getRssi()I
Searching for term: Landroid/net/NetworkCapabilities;-->hasTransport(I)Z!<=>!const/4 v1, 0x1
Searching for term: Landroid/net/ConnectivityManager;-->getNetworkCapabilities(Landroid/net/Network;)Landroid/net/NetworkCapabilities;!<=>!Landroid/net/NetworkCapabilities;-->hasTransport(I)Z
Searching for term: getNetworkCapabilities(Landroid/net/Network;)Landroid/net/NetworkCapabilities;!<=>!hasTransport(I)Z
```

|APK: ru.yandex.music:

```
|=====|
|Misc matches: 0|
|Cellular matches: 2|
|Device matches: 4|
|Proxy matches: 0|
|VPN matches: 5|
|WiFi matches: 6|
|-----|
```

|APK: ru.vtb24.mobilebanking.android:

```
|=====|
|Misc matches: 0|
|Cellular matches: 2|
|Device matches: 4|
|Proxy matches: 2|
|VPN matches: 4|
|WiFi matches: 6|
|-----|
```


APK: ru.megamarket.marketplace:	APK: ru.mts.mymts:	APK: com.ertelecom.agent:	APK: ru.beru.android:	APK: ru.vk.store:
Misc matches: 0	Misc matches: 0	Misc matches: 1	Misc matches: 0	Misc matches: 0
Cellular matches: 2	Cellular matches: 2	Cellular matches: 2	Cellular matches: 2	Cellular matches: 2
Device matches: 4	Device matches: 4	Device matches: 4	Device matches: 4	Device matches: 4
Proxy matches: 0	Proxy matches: 1	Proxy matches: 0	Proxy matches: 0	Proxy matches: 1
VPN matches: 3	VPN matches: 4	VPN matches: 3	VPN matches: 3	VPN matches: 3
WiFi matches: 6	WiFi matches: 6	WiFi matches: 6	WiFi matches: 6	WiFi matches: 6
APK: com.idamob.tinkoff.android:	APK: ru.alfabank.mobile.android:	APK: ru.vtb24.mobilebanking.andr	APK: com.google.android.odad:	APK: com.uma.musicvbk:
Misc matches: 0	Misc matches: 0	Misc matches: 0	Misc matches: 0	Misc matches: 0
Cellular matches: 2	Cellular matches: 2	Cellular matches: 2	Cellular matches: 0	Cellular matches: 2
Device matches: 4	Device matches: 4	Device matches: 4	Device matches: 1	Device matches: 4
Proxy matches: 0	Proxy matches: 0	Proxy matches: 2	Proxy matches: 0	Proxy matches: 1
VPN matches: 4	VPN matches: 3	VPN matches: 4	VPN matches: 0	VPN matches: 3
WiFi matches: 6	WiFi matches: 6	WiFi matches: 6	WiFi matches: 0	WiFi matches: 6
APK: com.vk.vkvideo:	APK: ru.rostel:	APK: ru.foodfox.client:	APK: ru.megafon.mlk:	APK: ru.oneme.app:
Misc matches: 0	Misc matches: 0	Misc matches: 0	Misc matches: 0	Misc matches: 0
Cellular matches: 2	Cellular matches: 2	Cellular matches: 2	Cellular matches: 2	Cellular matches: 2
Device matches: 4	Device matches: 4	Device matches: 4	Device matches: 4	Device matches: 4
Proxy matches: 1	Proxy matches: 1	Proxy matches: 0	Proxy matches: 0	Proxy matches: 0
VPN matches: 3	VPN matches: 3	VPN matches: 2	VPN matches: 2	VPN matches: 3
WiFi matches: 6	WiFi matches: 6	WiFi matches: 5	WiFi matches: 6	WiFi matches: 3
APK: com.topjohnwu.magisk:	APK: com.google.android.youtube:	APK: com.yandex.browser:	APK: ru.sberbankmobile:	APK: ru.zen.android:
Misc matches: 0	Misc matches: 0	Misc matches: 1	Misc matches: 0	Misc matches: 0
Cellular matches: 0	Cellular matches: 2	Cellular matches: 2	Cellular matches: 2	Cellular matches: 2
Device matches: 2	Device matches: 4	Device matches: 4	Device matches: 4	Device matches: 4
Proxy matches: 0	Proxy matches: 0	Proxy matches: 0	Proxy matches: 1	Proxy matches: 1
VPN matches: 2	VPN matches: 3	VPN matches: 5	VPN matches: 4	VPN matches: 3
WiFi matches: 0	WiFi matches: 6	WiFi matches: 6	WiFi matches: 6	WiFi matches: 6
APK: ru.yandex.music:	APK: ru.nspk.mirpay:	APK: ru.mail.mailapp:	APK: com.avito.android:	APK: ru.yandex.yandexmaps:
Misc matches: 0	Misc matches: 0	Misc matches: 0	Misc matches: 0	Misc matches: 0
Cellular matches: 2	Cellular matches: 1	Cellular matches: 2	Cellular matches: 2	Cellular matches: 2
Device matches: 4	Device matches: 4	Device matches: 4	Device matches: 4	Device matches: 4
Proxy matches: 0	Proxy matches: 0	Proxy matches: 1	Proxy matches: 0	Proxy matches: 1
VPN matches: 5	VPN matches: 2	VPN matches: 3	VPN matches: 3	VPN matches: 3
WiFi matches: 6	WiFi matches: 2	WiFi matches: 6	WiFi matches: 6	WiFi matches: 6
APK: ru.ok.android:	APK: ru.sbscs.store:	APK: com.wildberries.ru:	APK: com.vkontakte.android:	APK: ru.rutube.app:
Misc matches: 0	Misc matches: 0	Misc matches: 0	Misc matches: 0	Misc matches: 0
Cellular matches: 2	Cellular matches: 2	Cellular matches: 2	Cellular matches: 2	Cellular matches: 2
Device matches: 4	Device matches: 4	Device matches: 4	Device matches: 4	Device matches: 4
Proxy matches: 1	Proxy matches: 0	Proxy matches: 0	Proxy matches: 3	Proxy matches: 0
VPN matches: 3	VPN matches: 3	VPN matches: 3	VPN matches: 4	VPN matches: 3
WiFi matches: 6	WiFi matches: 6	WiFi matches: 6	WiFi matches: 6	WiFi matches: 6
APK: ru.yandex.taxi:	APK: ru.dublgis.dgismobile:	APK: ru.ozon.app.android:	APK: ru.kinopoisk:	
Misc matches: 0	Misc matches: 0	Misc matches: 0	Misc matches: 0	Misc matches: 0
Cellular matches: 2	Cellular matches: 2	Cellular matches: 2	Cellular matches: 2	Cellular matches: 2
Device matches: 4	Device matches: 4	Device matches: 4	Device matches: 4	Device matches: 4
Proxy matches: 1	Proxy matches: 1	Proxy matches: 2	Proxy matches: 0	Proxy matches: 0
VPN matches: 4	VPN matches: 6	VPN matches: 3	VPN matches: 5	VPN matches: 3
WiFi matches: 6	WiFi matches: 6	WiFi matches: 6	WiFi matches: 6	WiFi matches: 6

```

lachurc@lachurc-Nitro:~/Documents/school/CYBR470/project$ python3 scripts/resultsExtractor.py v
File 1: cellularF.txt
Search Term: Landroid/telephony/TelephonyManager; ->getNetworkOperatorName()Ljava/lang/String;
Result: ru.foodfox.client: 5
Result: com.uma.musicvk: 5
Result: ru.yandex.music: 4
Result: com.ertelecom.agent: 3
Result: ru.megamarket.marketplace: 6
Result: ru.vk.store: 8
Result: com.vk.vkvideo: 8
Result: ru.yandex.yandexmaps: 9
Result: ru.rutube.app: 4
Result: ru.vtb24.mobilebanking.android: 2
Result: ru.oneme.app: 3
Result: com.idamob.tinkoff.android: 7
Result: ru.alfabank.mobile.android: 2
Result: ru.dublgis.dgismobile: 8
Result: ru.beru.android: 5
Result: ru.mts.mymts: 8
Result: com.google.android.youtube: 1
Result: ru.ozon.app.android: 8
Result: com.avito.android: 7
Result: ru.mail.mailapp: 5
Result: com.wildberries.ru: 2
Result: ru.megafon.mlk: 1
Result: com.vkontakte.android: 13
Result: ru.rostel: 3
Result: ru.ok.android: 18
Result: ru.sberbankmobile: 4
Result: com.yandex.browser: 5
Result: ru.zen.android: 6
Result: ru.yandex.taxi: 6
Result: ru.sbcstore: 5
Result: ru.kinopoisk: 4
Search Term a: getSystemService(Ljava/lang/String)
Search Term b: phone
Search Term: Landroid/telephony/TelephonyManager;
Result: ru.sbcstore: 36
Result: com.vk.vkvideo: 50
Result: com.ertelecom.agent: 41
Result: ru.kinopoisk: 29
Result: ru.yandex.music: 29
Result: ru.megafon.mlk: 14
Result: com.avito.android: 39
Result: ru.ok.android: 95
Result: ru.rostel: 29
Result: ru.yandex.taxi: 59
Result: ru.vtb24.mobilebanking.android: 61
Result: com.uma.musicvk: 41
Result: com.idamob.tinkoff.android: 99
Result: com.google.android.youtube: 32
Result: ru.mail.mailapp: 66
Result: ru.ozon.app.android: 54
Result: ru.yandex.yandexmaps: 55
Result: ru.megamarket.marketplace: 44
Result: ru.alfabank.mobile.android: 31
Result: ru.foodfox.client: 38

```

```

lachurc@lachurc-Nitro:~/Documents/school/CYBR470/project$ python3 scripts/resultsExtractor.py
-----
APK: ru.yandex.music:
=====
Misc matches: 0
Cellular matches: 2
Device matches: 4
Proxy matches: 0
VPN matches: 5
WiFi matches: 6
-----
APK: ru.mts.mymts:
=====
Misc matches: 0
Cellular matches: 2
Device matches: 4
Proxy matches: 1
VPN matches: 4
WiFi matches: 6
-----
APK: ru.dublgis.dgismobile:
=====
Misc matches: 0
Cellular matches: 2
Device matches: 4
Proxy matches: 1
VPN matches: 6
WiFi matches: 6
-----
APK: com.uma.musicvk:
=====
Misc matches: 0
Cellular matches: 2
Device matches: 4
Proxy matches: 1
VPN matches: 3
WiFi matches: 6
-----
APK: ru.alfabank.mobile.android:
=====
Misc matches: 0
Cellular matches: 2
Device matches: 4
Proxy matches: 0
VPN matches: 3
WiFi matches: 6
-----
APK: com.google.android.youtube:
=====
Misc matches: 0
Cellular matches: 2
Device matches: 4
Proxy matches: 0

```

Conclusion

- 4 extra apps:
 - Мой Дом.py, odad systempackage, Youtube and Magisk
- **VPN matches:** 33/34 or 30/30
 - Youtube, Magisk and Мой Дом.py
- **Cellular matches:** 32/34 or 30/30
 - Youtube and Мой Дом.py
- **WiFi matches:** 32/34 or 30/30
 - Youtube and Мой Дом.py
- **Device matches:** 34/34 or 30/30
 - All the extra apps
- **Proxy matches:** 15/34 or 15/30
- **Misc matches:** 2/34 or 2/30(Tor was the search here)

Future works

- Create machine learning model - likely a PLA or neural net based upon information found, with some fingerprints being given more weights.
- Develop more efficient searching method
 - Not relying on ripgrep
 - Have possibility for n search terms in single fingerprint (not just one)
- Increase sample size, go from the original 30 to testing it out on all 463 affected Russian apps, possibly testing on Chinese and American apps too.
- Use a proxy Access point with computer and record all traffic to see where information is being sent to.
 - Possibly spoofing values to see difference in behaviour